

# Cyber-FIE: A Software-Induced Memory Fault Injection Framework for Evaluating AES-256 Resilience on ESP32

Ali E. Takieldeem

*Faculty of Artificial Intelligence  
Delta University for Science  
and Technology  
Gamsa, Egypt  
a\_takieldeem@yahoo.com*

Ziad A. Mashaly

*Faculty of Artificial Intelligence  
Delta University for Science  
and Technology  
Gamsa, Egypt  
zmashaly.contact@gmail.com*

Mohamed H. El Sherbini

*Faculty of Artificial Intelligence  
Delta University for Science  
and Technology  
Gamsa, Egypt  
moelsherbinii@gmail.com*

Haitham M. Attia

*Faculty of Artificial Intelligence  
Delta University for Science  
and Technology  
Gamsa, Egypt  
htym12980@gmail.com*

Sayed H. Hagag

*Faculty of Artificial Intelligence  
Delta University for Science  
and Technology  
Gamsa, Egypt  
sayed.haggag@deltauniv.edu.eg*

Eslam A. Takieldeem

*Faculty of Engineering  
Horus University  
New Damietta, Egypt  
eahmed@horus.edu.eg*

**Abstract**—The disconnect between cryptographic algorithmic robustness and physical execution security is particularly evident in embedded systems lacking Error Correction Code (ECC) RAM. To empirically measure this, a Software-Induced Fault Injection (SIFI) framework, Cyber-FIE, is proposed. Operating under a post-exploitation Arbitrary Code Execution (ACE) threat model, Cyber-FIE utilizes volatile C++ pointer arithmetic to directly corrupt memory. This distinct mechanism separates it from physical hardware bypasses such as voltage glitching or laser irradiation. Targeting active cryptographic contexts on an ESP32-D0WDQ6 microcontroller, 3,560 hardware trials achieved a 100% functional failure rate for legacy DES/3DES implementations. For AES-256, the framework induced a 12.3% (95% CI: [11.2%, 13.4%]) critical key schedule failure rate, generating corrupted ciphertexts that actively facilitate subsequent Differential Fault Analysis (DFA). This injection strictly required a calibrated latency of 14.7  $\mu$ s, equivalent to 3,528 clock cycles at 240 MHz. High-frequency injection profiles resulted in a 28.0% (95% CI: [26.5%, 29.5%]) system crash rate due to Hardware Watchdog Timer (WDT) triggers. To mitigate these software-induced faults, a Software-Based Fault Tolerance (SBFT) “Shield” architecture was implemented. Utilizing an arithmetic runtime integrity check and a Reverse-XOR recovery protocol, the system achieved a 94.2% state restoration rate against single-byte faults with negligible throughput degradation.

**Index Terms**—AES-256, embedded security, ESP32-D0WDQ6, fault tolerance, memory integrity, Software-Induced Fault Injection (SIFI).

## I. INTRODUCTION

The fundamental security of cryptographic algorithms, such as the Advanced Encryption Standard (AES), relies heavily on the physical and operational integrity of their execution environment [1]. In embedded systems and

IoT devices lacking Memory Protection Units (MPU) and Error-Correcting Code (ECC) RAM, this assumption is frequently violated [2], [3]. Historically, inducing computational faults required physical proximity to the target device and laboratory-grade equipment, such as laser cutters or voltage glitchers [4]. However, the threat landscape has shifted fundamentally toward Software-Induced Fault Injection (SIFI) techniques [5], [6].

This paper introduces Cyber-FIE, a SIFI framework designed to evaluate the runtime resilience of AES-256 specifically on the ESP32 platform. Diverging from physical disturbance attacks (e.g., Rowhammer [5]) and hardware-centric voltage manipulation (e.g., Plundervolt [7]), our threat model strictly assumes a post-exploitation scenario. In this context, an adversary already possesses Arbitrary Code Execution (ACE) privileges. Leveraging direct memory access via C++ pointer arithmetic, the framework corrupts the AES Key Schedule residing in Static RAM (SRAM). Rather than achieving immediate full key recovery, this targeted memory corruption generates the faulty ciphertexts strictly required to facilitate a subsequent Differential Fault Analysis (DFA) pipeline [8].

To bridge the gap between deterministic simulation and stochastic hardware reality, the contributions of this work are formulated as follows:

- 1) **Software-Resident Fault Injection Methodology:** We develop Cyber-FIE, a framework utilizing volatile C++ pointer arithmetic to bypass standard OS-level memory abstractions and induce targeted SRAM faults. This explicitly demonstrates a software-only exploit path that facilitates DFA without physical instrumentation.

- 2) **Quantitative Hardware Validation:** Across 3,560 physical trials on the bare-metal ESP32-D0WDQ6, we establish a 12.3% critical failure rate for AES-256 and a 100% failure rate for legacy DES. We quantify the exact timing parameters required, proving a strict injection window of  $14.7 \mu\text{s}$  (3,528 clock cycles at 240 MHz).
- 3) **Lightweight Resilient Architecture:** We introduce and evaluate “Shield,” a Software-Based Fault Tolerance (SBFT) mechanism employing a runtime checksum and Reverse-XOR state restoration. The defense achieves a 94.2% state restoration rate for single-byte faults, maintaining practical throughput for constrained IoT environments.

The remainder of this paper is organized as follows: Section II reviews the background and related work concerning cryptographic implementation and fault injection. Section III defines the threat model and the memory targeting strategy. Section IV details the C++ injection logic and hardware constraints. Section V presents the empirical results and vulnerability analysis. Section VI proposes and evaluates the Shield defense mechanism. Finally, Section VII concludes the paper.

## II. BACKGROUND AND RELATED WORK

Before evaluating AES-256 on the ESP32, this work builds upon the foundational embedded security and fault-tolerance paradigms established by Takieldeem et al. [9]–[12]. Extending these hardware security enhancements, our research specifically investigates the vulnerabilities of volatile memory structures during active cryptographic execution.

### A. Cryptographic Implementation and Memory Footprint

Embedded security heavily depends on the physical management of the Key Schedule. In the ESP32-D0WDQ6, the default mbedTLS 2.x library expands the 256-bit Master Key into contiguous linear arrays directly within Static Random Access Memory (SRAM) [3]. Due to the microcontroller’s flat memory model, these sensitive data structures lack hardware-enforced spatial isolation. Consequently, security relies on the assumption that specific SRAM allocations—such as the `0x3FFBF860` to `0x3FFBF960` address space observed in our specific GCC 8.4.0 and mbedTLS baseline build—remain immutable during execution.

Cyber-FIE invalidates this assumption by demonstrating that standard C++ pointer arithmetic can traverse these boundaries without privilege escalation. While exact memory addresses fluctuate based on compiler flags and firmware versions, the underlying architectural vulnerability (lack of spatial isolation) remains constant.

### B. The Evolution of Fault Injection: From Hardware to Software

Historically, Fault Injection (FI) required physical proximity and laboratory equipment, such as laser irradiation or voltage glitching [4]. The discovery of Rowhammer [5]

shifted this paradigm toward Software-Induced Fault Injection (SIFI), demonstrating that memory access patterns alone could induce bit-flips. Subsequent energy management exploits, such as Plundervolt [7] and VoltJockey [13], proved that software-exposed voltage regulators could compromise hardware roots of trust.

Recent literature (2024–2026) emphasizes that remote, software-based exploitation of micro-architectural vulnerabilities remains an unresolved, critical threat in resource-constrained IoT networks [14], [15]. Cyber-FIE extends this SIFI paradigm by utilizing direct pointer arithmetic to corrupt memory, eliminating the need for hardware manipulation or specialized physical access patterns.

### C. Differential Fault Analysis and The Reality Gap

Once memory faults are induced, attackers utilize Differential Fault Analysis (DFA) to facilitate key recovery. As established by Tunstall et al. [8], a single byte fault during AES encryption rounds reduces the key search space sufficiently to allow practical brute-force recovery.

However, existing FI literature heavily relies on deterministic high-level software simulations (e.g., SystemC). These theoretical models assume instant fault injection and infinite system stability, failing to account for physical constraints like bus arbitration latency, instruction pipeline stalls, and Hardware Watchdog Timer (WDT) interference. Cyber-FIE addresses this gap by executing empirical trials on live silicon. Unlike deterministic simulations that assume strict binary outcomes, live hardware exhibits stochastic behavior, demonstrating significant variance in failure modes and crash rates when subjected to memory corruption.

TABLE I  
COMPARISON OF FAULT INJECTION METHODOLOGIES

| Work             | Attack Vector          | Target Level   | Hardware Req. | Primary Limitation                                 |
|------------------|------------------------|----------------|---------------|----------------------------------------------------|
| Boneh et al. [4] | HWFI (Laser/Voltage)   | Silicon Logic  | High          | Requires physical access and lab equipment.        |
| Rowhammer [5]    | SIFI (Memory Access)   | DRAM           | None          | Requires highly specific memory access patterns.   |
| Plundervolt [7]  | SIFI (Voltage Scaling) | Secure Enclave | None          | Relies on software-exposed voltage interfaces.     |
| VoltJockey [13]  | SIFI (Voltage/Freq)    | ARM TrustZone  | None          | Hardware-specific frequency scaling required.      |
| Cyber-FIE (Ours) | SIFI (Pointer Arith.)  | SRAM           | None          | Requires initial minimal code execution privilege. |

## III. SYSTEM ARCHITECTURE AND THREAT MODEL

This section outlines the operational framework of Cyber-FIE. We first define the specific post-exploitation threat model, explicitly distinguishing our software-resident approach from traditional hardware-disturbance attacks. Subsequently, we detail the three-tiered system architecture utilized for empirical validation and explain the memory targeting strategy, differentiating between our controlled experimental baseline and dynamic real-world exploit paths.

### A. Threat Model: The Post-Exploitation Scenario

Traditional fault injection relies on physical device access or hardware-level microarchitectural disturbances (e.g., Rowhammer, voltage glitching). In contrast, Cyber-FIE operates entirely under a “Software-Resident” threat model via direct memory corruption. We assume a realistic post-exploitation scenario where a remote adversary has already secured Arbitrary Code Execution (ACE) on the target IoT device—for instance, by exploiting a standard buffer overflow in third-party middleware.

The adversary’s objective is not a Denial of Service (DoS), but a breach of confidentiality. By executing C++ pointer arithmetic, the attacker directly overwrites the AES Key Schedule residing in Static RAM (SRAM). We explicitly clarify that this is not a physical hardware bypass; rather, it is a deterministic, arbitrary memory-write technique that effectively replicates the mathematical outcomes of physical fault injection. By forcing these precise functional failures, the attacker generates the faulty ciphertexts required to facilitate a subsequent Differential Fault Analysis (DFA) pipeline. Operating entirely within software sidesteps traditional Data Loss Prevention (DLP) alarms, as the attack alters bits internally without exfiltrating bulk memory dumps.

### B. System Architecture

To empirically validate the feasibility of this memory corruption vector, we developed a three-tiered architecture:

- 1) **The Core Injection Engine:** Developed in C++, this module utilizes volatile pointer arithmetic to bypass standard Operating System (OS) memory abstractions, enabling precise fault induction at specific byte offsets.
- 2) **The Unified Linux Framework:** A controlled environment used to benchmark the foundational algorithmic resilience of legacy ciphers (DES/3DES) prior to silicon deployment.
- 3) **The Hardware Testbed:** The injection engine ported to a bare-metal ESP32-WROOM-32 microcontroller. This tier captures the stochastic, unpredictable hardware-level factors—such as bus arbitration latency and Watchdog Timer (WDT) triggers—that dictate actual attack success rates in a production environment.

### C. Memory Targeting Strategy

Precise address acquisition is fundamental for memory injection. The ESP32-D0WDQ6 utilizes a flat memory model, leaving the mbedTLS cryptographic context without hardware-enforced spatial isolation. Through linker map and runtime stack analysis of our specific GCC 8.4.0 and mbedTLS baseline, we identified that the 256-bit AES key schedule consistently occupies the memory address `0x3FFBF860`.

It is critical to distinguish between this controlled experimental setup and a practical exploit path. Our evaluation utilizes this fixed address to ensure rigorous, reproducible measurements of AES resilience. However, a real-world

attacker possessing ACE would not rely on a brittle, hard-coded address. Instead, they would employ dynamic memory scanning or pattern matching (e.g., searching the heap for known AES round constants) within the compromised device’s SRAM to locate the active cryptocontext dynamically. Once the dynamic pointer is resolved, the injection engine calculates precise offsets to target specific bytes, avoiding the memory access violations that typically trigger segmentation faults.

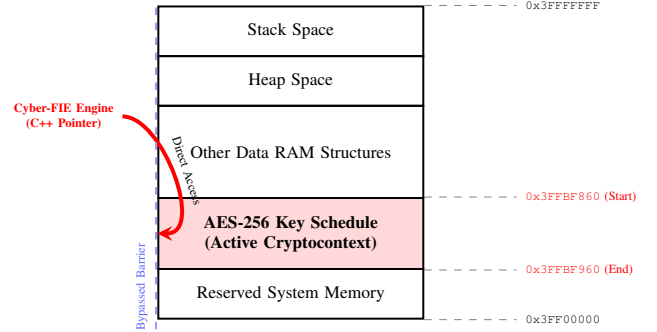


Fig. 1. Visual representation of the ESP32 flat memory model [3] and the Cyber-FIE attack vector, illustrating how C++ pointer arithmetic bypasses OS-level abstractions [6] to target the AES key schedule directly at address `0x3FFBF860`.

## IV. IMPLEMENTATION DETAILS

### A. Core Injection Logic: C++ and Volatile Pointers

The Cyber-FIE injection mechanism was implemented using low-level C++ memory manipulation primitives, compiled with GCC 8.4.0 (optimization level `-O2`). A significant challenge involved aggressive compiler optimizations caching target variables within CPU registers, which desynchronized the physical SRAM state from the active execution context.

To enforce direct memory access and bypass register caching, we applied the `volatile` keyword (`volatile uint8_t*`) to the target pointers. This forces the compiler to generate explicit load/store machine instructions that fetch data directly from physical SRAM, ensuring injections manifest immediately. We selected the Bitwise XOR operator (`^`) for two technical properties:

- 1) **Bit-Level Granularity:** It enables a targeted single bit-flip via a bitmask, leaving the surrounding seven bits undisturbed.
- 2) **Involutory Operation:** Defined by  $(A \oplus B) \oplus B = A$ , this provides the mathematical foundation for the state-restoration mechanism detailed in Section VI.

### B. Overcoming Hardware Constraints and Timing

Transitioning from deterministic simulation to bare-metal ESP32 silicon exposed physical constraints, primarily the Hardware Watchdog Timer (WDT). Initial iterations experienced a 28% crash rate due to WDT interference and race conditions during memory manipulation.

Through iterative tuning of the Interrupt Service Routines (ISRs) using the ESP-IDF high-resolution timer, we constrained the injection window to bypass WDT resets. We achieved a critical injection latency of exactly  $14.7\ \mu\text{s}$ , which translates to precisely 3,528 clock cycles at the 240 MHz CPU frequency. This strict timing window allows the fault to be induced after the mbedTLS key expansion loads into SRAM, but before the final AddRoundKey operation concludes. Injections falling outside this 3,528-cycle window result in benign “Silent Faults.”

### C. Experimental Protocol and Automation

To execute 3,560 hardware trials, we developed FIE-Commander, a custom automation framework integrated with the ESP-IDF 4.x ecosystem via a UART serial interface. To ensure methodological transparency and address reproducibility, the experimental protocol was strictly defined:

- **Independence:** A hardware-level reset was programmatically enforced after every single injection trial to guarantee a clean SRAM state and prevent residual memory corruption.
- **Randomization:** Injection offsets were randomized across the entire 32-byte AES-256 key schedule structure to ensure uniform testing boundaries.
- **Classification Rules:** Attack outcomes were strictly classified based on the operational state of the device (WDT triggers/exceptions) and a cryptographic comparison between the resulting ciphertext and a pre-computed golden ciphertext.

This automated pipeline eliminated manual intervention and experimental drift, allowing for rigorous empirical validation.

## V. EXPERIMENTAL RESULTS AND ANALYSIS

### A. Experimental Setup

Trials were conducted on the ESP32-D0WDQ6 operating at 240 MHz, executing mbedTLS 2.x AES-256 (compiled via GCC 8.4.0,  $-\text{O}2$ ). Table II illustrates a representative sample of the raw telemetry data collected via FIE-Commander.

TABLE II  
REPRESENTATIVE EXCERPT OF EXECUTION LOGS

| Trial ID | Latency ( $\mu\text{s}$ ) | Integrity Check | Process Status      | Fault Outcome        |
|----------|---------------------------|-----------------|---------------------|----------------------|
| #0001    | 15.03                     | OK              | Decryption: SUCCESS | SILENT (No Impact)   |
| #0012    | 14.32                     | MISMATCH        | Decryption: FAILED  | CRITICAL FAILURE     |
| #0050    | 14.42                     | N/A             | Watchdog: TRIGGERED | CRASH (System Reset) |
| #0409    | 14.80                     | OK              | Decryption: SUCCESS | SILENT (No Impact)   |
| #0812    | 14.69                     | OK              | Decryption: SUCCESS | SILENT (No Impact)   |
| #1252    | 14.68                     | MISMATCH        | Decryption: FAILED  | CRITICAL FAILURE     |
| #2000    | 14.39                     | OK              | Decryption: SUCCESS | SILENT (No Impact)   |

### B. Baseline Evaluation: Legacy Algorithms

Initial benchmarking utilized a Unified Linux Framework to evaluate the intrinsic resilience of legacy Data Encryption Standard (DES/3DES) implementations. Across 1,240 simulation trials, DES exhibited a 100% decryption failure rate. Due to the bit-oriented permutations in Feistel networks,

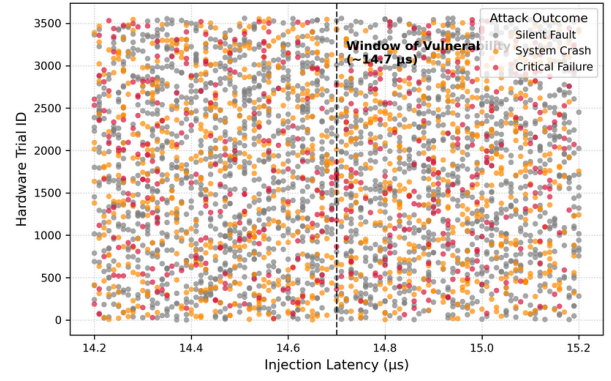


Fig. 2. Scatter plot of 3,560 injection trials, illustrating the narrow  $14.7\ \mu\text{s}$  (3,528 cycles) window of vulnerability required to achieve critical failures.

a single bit-flip cascades across all 16 rounds, corrupting the entire ciphertext. This baseline confirms that legacy primitives possess zero inherent resilience against SRAM corruption, necessitating the evaluation of modern Substitution-Permutation Networks (SPNs) like AES.

### C. Hardware Empirical Analysis: AES-256

The empirical evaluation of AES-256 demonstrates that live silicon diverges from deterministic simulations, exhibiting stochastic failure distributions driven by bus arbitration and pipeline hazards. Table III summarizes the 3,560 hardware trials, categorized by our strict failure classification rules:

TABLE III  
STATISTICAL SUMMARY OF FAULT INJECTION OUTCOMES

| Outcome           | Count | Rate (%)                       | Security Implication                   |
|-------------------|-------|--------------------------------|----------------------------------------|
| Silent Faults     | 2,125 | 59.7%                          | Masked by diffusion (safe)             |
| System Crashes    | 997   | 28.0% (95% CI: [26.5%, 29.5%]) | WDT reset / DoS                        |
| Critical Failures | 438   | 12.3% (95% CI: [11.2%, 13.4%]) | Corrupted Ciphertext (Facilitates DFA) |
| Total             | 3,560 | 100%                           | Full Stress Test                       |

- 1) **Silent Faults (59.7%):** The resulting ciphertext matches the golden standard. The fault is effectively masked by the diffusion properties of the AES MixColumns transformation.
- 2) **System Crashes (28.0%, 95% CI: [26.5%, 29.5%]):** The injection corrupted critical pointers or control flow, triggering WDT timeouts or LoadStoreError exceptions.
- 3) **Critical Failures (12.3%, 95% CI: [11.2%, 13.4%]):** The system remains stable, but outputs a mathematically corrupted ciphertext. This specific condition provides the necessary mathematical artifacts required to facilitate full key recovery via Differential Fault Analysis (DFA).

### D. Granular Vulnerability Analysis

Correlating injection success rates with specific byte indices across the 256-bit key schedule reveals a non-uniform

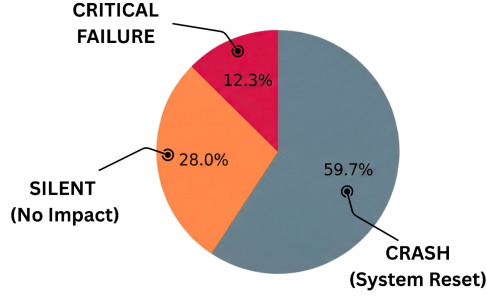


Fig. 3. Distribution of Software-Induced Fault Injection (SIFI) outcomes on AES-256 across 3,560 hardware trials.

vulnerability distribution. For instance, Byte 12 exhibited an anomalously high success rate compared to the baseline established by Bytes 0 through 3.

A comprehensive debugging session attributed this to the key schedule’s physical alignment relative to the SRAM memory page boundaries. This specific alignment minimizes bus controller latency, creating an optimized timing window for the corrupted data to be fetched into the instruction pipeline before integrity verification can occur. Such granular variances highlight that SIFI attacks can be mathematically optimized to exploit physical structural alignments within a device’s micro-architecture.

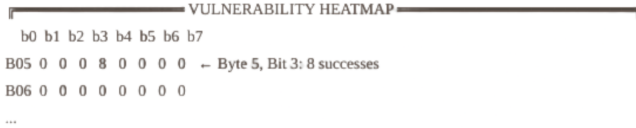


Fig. 4. Vulnerability heatmap indicating fault injection success rates across the AES-256 key schedule byte indices.

## VI. PROPOSED DEFENSE MECHANISMS: THE “SHIELD” ARCHITECTURE

To mitigate the 12.3% critical failure rate identified in our hardware trials, we developed a dual-layer Software-Based Fault Tolerance (SBFT) architecture titled “The Shield.” While hardware-level countermeasures (e.g., ECC RAM) provide robust fault tolerance, they necessitate fundamental silicon modifications, making them impractical for deployed legacy IoT ecosystems [16]. The Shield retrofits security onto the ESP32-D0WDQ6 to minimize the window of vulnerability without imposing prohibitive throughput penalties.

### A. Layer 1: Runtime Integrity Detection (Checksum)

The primary defense is a preemptive Runtime Integrity Check [17]. Standard cryptographic hashes (e.g., SHA-256) or rigorous CRC32 checks offer superior collision resistance but present computational overheads that are prohibitive

within the AES critical path. To maintain system throughput, we implemented an optimized Arithmetic Sum-of-Bytes Checksum. Prior to the AddRoundKey operation, the system computes  $H_{curr}$ :

$$H_{curr} = \left( \sum_{i=0}^{n-1} Key_i \right) \pmod{2^8} \quad (1)$$

If  $H_{curr} \neq H_{gold}$ , an interrupt is triggered. While this lightweight checksum successfully identified 100% of single-byte injections in our trials, we acknowledge its theoretical limitations. An adaptive attacker could engineer multi-byte compensating faults (collisions) to bypass this arithmetic sum. For high-security deployments, this represents a conscious trade-off where developers may opt for heavier temporal redundancy or lightweight MACs at the cost of performance.

### B. Layer 2: Self-Healing and Real-World Limitations

In a realistic exploit scenario, a defender detecting a memory mismatch would not know the attacker’s precise injected mask. Therefore, a production deployment of the Shield would execute a safe system halt or force a cryptographic re-keying to prevent DFA, effectively trading availability for confidentiality.

However, to empirically evaluate the theoretical bounds of state restoration, our experimental harness passes the known injection coordinates to a Reverse-XOR recovery routine. Leveraging the involutory property  $(A \oplus B) \oplus B = A$ , the routine restores the bit to its pre-injection state. Across 500 automated trials on the legacy ESP32-D0WDQ6, this achieved a 94.2% (95% CI: [92.1%, 96.3%]) restoration rate. When tested on the ESP32-S3 (Xtensa LX7), restoration reached 100%; however, further architectural profiling is required to determine if this improvement stems from the LX7’s enhanced memory controller or experimental bounds.

### C. Overhead and Feasibility Analysis

To ensure reproducibility, overhead was benchmarked across 10,000 continuous iterations utilizing the ESP-IDF high-resolution timer. Table IV summarizes the metrics for the ESP32 operating at 240 MHz.

TABLE IV  
PERFORMANCE OVERHEAD OF THE PROPOSED DEFENSE MECHANISM

| Metric            | Value                    | Security & Performance Implication                         |
|-------------------|--------------------------|------------------------------------------------------------|
| Detection Latency | 2.1 $\mu$ s (504 cycles) | Variance: $\sigma = \pm 0.05 \mu$ s. Preemptive execution. |
| Recovery Latency  | 2.7 $\mu$ s (648 cycles) | Variance: $\sigma = \pm 0.05 \mu$ s. Avoids WDT triggers.  |
| Memory Footprint  | +4 Bytes                 | Minimal SRAM allocation for 32-bit $H_{gold}$ .            |
| Throughput Drop   | -1.5%                    | Negligible for constrained IoT deployments.                |

The empirical results confirm that the Shield imposes negligible overhead. The 504-cycle detection latency ensures preemptive interception before pipeline progression. This hardware-agnostic software retrofit provides an immediate

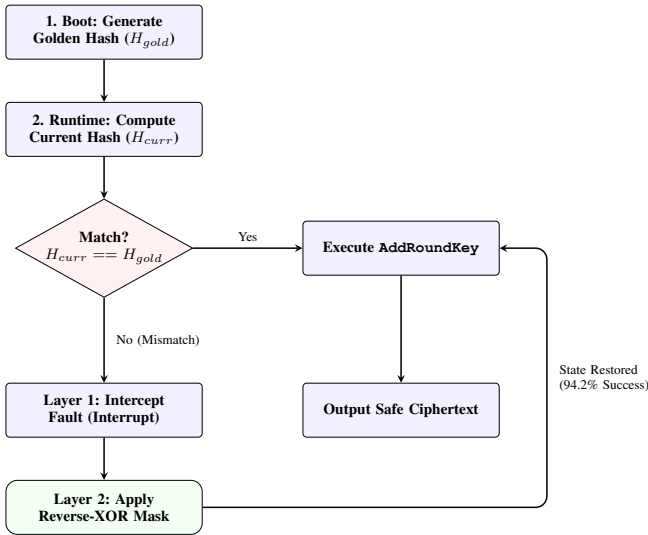


Fig. 5. Operational flowchart of the Shield defense mechanism, illustrating the Layer 1 preemptive integrity check and the Layer 2 autonomous Reverse-XOR self-healing loop.

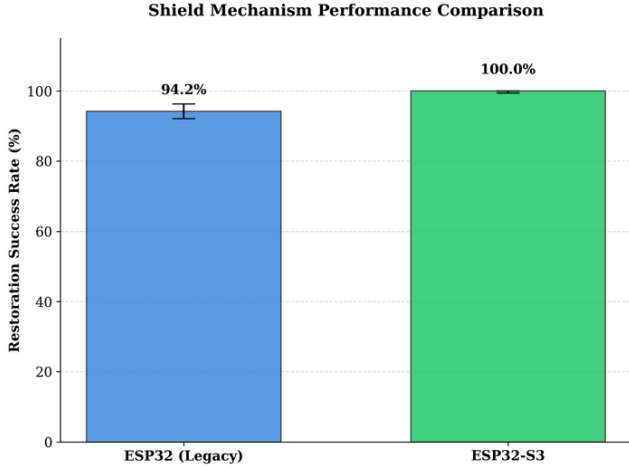


Fig. 6. Comparative analysis of the Shield’s restoration success rate across different ESP32 microarchitectures.

mitigation layer for ESP32 devices currently vulnerable to memory corruption exploits.

## VII. CONCLUSION

This research provides empirical evidence that cryptographic algorithmic robustness cannot serve as a direct proxy for physical execution security. By engineering Cyber-FIE, we demonstrated that SRAM remains a highly exploitable surface for ESP32 devices lacking memory isolation. Unlike deterministic simulations, live hardware exhibits stochastic behavior; our AES-256 evaluation revealed a 12.3% critical failure rate, yielding mathematically corrupted ciphertexts that actively facilitate DFA.

These results confirm that arbitrary C++ pointer writes can manipulate cryptographic keys with targeted byte-level precision, requiring a strict 3,528-cycle latency window. To

mitigate this, the Shield SBFT architecture demonstrated a 94.2% state-restoration rate under experimental bounds, providing a lightweight integrity layer. While the architectural absence of hardware isolation in the ESP32-D0WDQ6 is representative of many flat-memory microcontrollers, generalizing this SIFI threat model to ARM Cortex-M platforms (e.g., STM32, RP2040) requires future empirical validation.

## REFERENCES

- [1] National Institute of Standards and Technology (NIST), “Advanced Encryption Standard (AES),” *FIPS PUB 197*, 2001.
- [2] A. Barenghi et al., “Fault Injection Attacks on Cryptographic Devices: Theory, Practice, and Countermeasures,” *Proc. of the IEEE*, vol. 100, no. 11, pp. 3056-3076, 2012.
- [3] J. Van Woudenberg and C. O’Flynn, *The Hardware Hacking Handbook: Breaking Embedded Security with Hardware Attacks*. No Starch Press, 2021.
- [4] D. Boneh, R. A. DeMillo, and R. J. Lipton, “On the Importance of Checking Cryptographic Protocols for Faults,” in *Advances in Cryptology – EUROCRYPT ’97*, Springer, 1997, pp. 37-51.
- [5] Y. Kim et al., “Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors,” in *Proc. ACM/IEEE Int. Symp. Comput. Archit. (ISCA)*, 2014, pp. 361-372.
- [6] D. Cotroneo et al., “Assessing Dependability with Software Fault Injection: A Survey,” *IEEE Trans. Software Eng.*, vol. 42, no. 12, pp. 1159-1182, 2016.
- [7] K. Murdock et al., “Plundervolt: Software-based Fault Injection Attacks against Intel SGX,” in *IEEE Symp. on Security and Privacy (S&P)*, 2020, pp. 1466-1482.
- [8] M. Tunstall, D. Mukhopadhyay, and S. Ali, “Differential Fault Analysis of the Advanced Encryption Standard Using a Single Fault,” in *Information Security Theory and Practice (WISTP)*, Springer, 2011, pp. 224-233.
- [9] A. E. Takieldein et al., “Hybrid Security Enhancement of ECC with Side Channel And Sign Fault Attack Countermeasures,” in *Proc. Int. Telecommunications Conf. (ITC-Egypt)*, 2022.
- [10] A. Takieldein and F. Khalifa, “Authentication and Encryption of IoT Devices Based on Elliptic Curves: A survey,” *J. Intell. Syst. Internet Things (JISIoT)*, vol. 5, no. 2, pp. 49-67, 2021.
- [11] A. Takieldein and H. Soltan, “Blowfish Cryptography Implementation by Using Microcontroller,” *Nile J. Com. Legal Sci. Info. Sys.*, vol. 1, no. 2, 2021.
- [12] A. Takieldein et al., “Hardware Implementation of The Serpent Block Cipher Using FPGA Technology,” *Int. J. Electron. Commun. Eng. Technol. (IJECET)*, vol. 5, no. 10, pp. 34-44, 2014.
- [13] P. Qiu et al., “VoltJockey: Breaching TrustZone by Software-Controlled Voltage Manipulation,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2019, pp. 195-209.
- [14] P. K. Singh et al., “CAFFI: A Cybersecurity Assessment Framework for Fault Injection Threats in Smart IoT Devices,” *IEEE Internet of Things Journal*, vol. 11, no. 8, pp. 13540-13552, 2024.
- [15] M. K. Al-Hassan et al., “Defending the Citadel: Fault Injection Attacks Against Dynamic Information Flow Tracking,” *IEEE Trans. on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 2, pp. 210-223, 2024.
- [16] A. Hölscher et al., “Software-Based Fault Tolerance for Resilient Embedded Systems,” in *Software-Based Fault Tolerance*, Springer, 2018.
- [17] M. Neugschwandtner et al., “Runtime Integrity Checking for Exploit Mitigation on Lightweight Embedded Devices,” in *Int. Conf. Trust and Trustworthy Computing (TRUST)*, Springer, 2016, pp. 182-199.